# Table of Contents

# 1 Executive Summary

**CRITICAL FINDING**

This project is a DASHBOARD that SIMULATES an Epistemic Runtime. It is NOT a working implementation. Specification implementation stands at approximately 8%. The .epd DSL evaluator is the only component that qualifies as real, kernel-quality code. Everything else is architectural only, contradicts the specification, or simply does not exist.

This convergence audit was conducted against the Epistemic Runtime v0.8 specification to determine whether the current repository implementation is converging toward the specified architecture. The findings are unequivocal: it is not. The repository contains a well-constructed browser dashboard with 37 React components and 18 API routes, but the underlying kernel -- the acceptance pipeline, the fact log, the deterministic sequencer, the projection engine -- does not exist. What exists is a visual simulation that reads from and writes to SQLite via Prisma with no intermediate processing layer whatsoever.

## Key Metrics

| Metric | Value | Assessment |
|---|---|---|
| Spec Implementation | ~8% | CRITICAL |
| Dashboard Completeness | ~90% | GOOD |
| Kernel Completeness | ~5% | CRITICAL |
| API Routes (Real) | 11/15 | MODERATE |
| API Routes (Mock) | 3/15 | WARNING |
| API Routes (Placeholder) | 1/15 | CRITICAL |
| Non-determinism Instances | 8 CRITICAL | CRITICAL |
| Spec Contradictions | 3 | CRITICAL |
| Test Coverage | 0% | CRITICAL |

## Three Critical Contradictions

The audit has identified three areas where the implementation directly contradicts the v0.8 specification, rather than merely falling short of it. These are not gaps or incomplete implementations; they are active contradictions that must be resolved before any convergence can occur.

- Hash Algorithm: The specification mandates SHA-256 for all content-addressed hashing. The implementation uses FNV-1a 32-bit, a non-cryptographic hash with known collision properties at scale.

This is not a preference difference; it is a fundamental architectural divergence that affects data integrity, proof verification, and content addressing throughout the system.

- Merkle Structure: The specification requires a Merkle Mountain Range (MMR) for efficient append-only proofs and historical verification. The implementation uses a simple binary Merkle tree that is rebuilt on every insertion, losing all historical proof capability. This contradicts the append-only, incrementally verifiable nature of the specification.
- Zero-Knowledge Proofs: The specification describes ZK proof generation for policy compliance verification. The implementation generates random hexadecimal strings using `Math.random().toString(16)` and presents them as proofs. These are fabricated values with zero cryptographic validity. Deploying this would constitute a security vulnerability.

The single bright spot is the .epd DSL engine, which includes a real tokenizer, parser, validator, evaluator, and self-repair mechanism. This code is well-structured, spec-aligned, and represents genuine kernel-quality implementation. It is, however, the only such component in the entire repository.

# 2 Repository Architecture

The repository is structured as a Next.js 15 application with a Prisma ORM layer backed by SQLite. The frontend consists of 37 React components organized under `src/components/epistemic/`, each representing a section of the dashboard. The backend exposes 18 API routes under `src/app/api/`. The only standalone kernel module is the EPD engine located at `src/lib/epd/`.

## Architecture Graph

```
Browser (React/Next.js) | +--> 18 API Routes | | | +--> 11 REAL routes --> Prisma ORM --> SQLite | +--> 3 MOCK routes --> Hardcoded data | +--> 1 PLACEHOLDER --> Empty handler | +--> 3 Hybrid routes --> Prisma + mock fallback | +--> EPD Engine (standalone) tokenizer --> parser --> validator --> evaluator --> self-repair
```

## API Route Classification

Each API route was audited to determine whether it performs real data operations, returns hardcoded mock data, or is a placeholder with no implementation. The classification reveals that while most routes connect to Prisma for database operations, none of them route through an acceptance pipeline or perform the canonicalization, verification, or signature checks specified in the v0.8 architecture.

| Category | Count | Routes | Assessment |
|---|---|---|---|
| REAL (Prisma) | 11 | policies, policies/[id], shards, search, export, merges, merges/simulate, audit, metrics, system, convergence | CRUD only, no kernel |
| MOCK (Hardcoded) | 3 | proofs, trust-runtime, shadow-bridge | No real data |
| PLACEHOLDER | 1 | acceptance-engine | Empty handler |
| Hybrid | 3 | architecture, timeline, migration | Partial Prisma + mock |

## The EPD Engine

The EPD (Epistemic Policy Definition) engine is the only component that qualifies as genuine kernel-quality code. Located at `src/lib/epd/`, it consists of five modules that form a complete DSL processing pipeline:

- Tokenizer (`tokenizer.ts`): Lexes .epd source into typed tokens including keywords, operators, literals, and identifiers. Handles comments and whitespace.
- Parser (`parser.ts`): Constructs an abstract syntax tree from the token stream. Supports policy definitions, rule blocks, conditions, and actions.

- Validator (`validator.ts`): Performs semantic validation on the AST, checking for type errors, undefined references, and constraint violations.
- Evaluator (`index.ts`): Executes validated policies against runtime fact contexts. Returns structured evaluation results with match status and computed values.
- Self-Repair: When evaluation fails due to schema drift or missing fields, the engine attempts automated repair by applying heuristics to reconcile the policy with the current fact schema.

The critical issue is that there is no kernel connecting these components to the broader system. All data flows are direct CRUD operations through Prisma. There is no acceptance pipeline mediating writes, no fact log recording state transitions, and no deterministic sequencer ordering events. The EPD engine operates in isolation, evaluating policies that have no enforced effect on the data layer.

# 3 Specification Mapping Matrix

The following matrix maps each component defined in the Epistemic Runtime v0.8 specification to its current implementation status. Each component is assessed on two axes: Architecture Status (whether the component's architecture exists in the codebase) and Implementation Status (whether the component is actually implemented and functional). The Readiness percentage represents the combined assessment of both axes.

| Spec Component | Architecture | Implementation | Ready | Notes |
|---|---|---|---|---|
| Acceptance Pipeline | Architectural Only | NOT IMPLEMENTED | 5% | No acceptance pipeline exists. Writes go directly to SQLite. |
| Fact Log (Event Store) | Architectural Only | NOT IMPLEMENTED | 5% | No event sourcing. Prisma schema has no event/fact table. |
| Deterministic Sequencer | Architectural Only | NOT IMPLEMENTED | 0% | No sequencer. All ordering is database-default. |
| Projection Engine | Architectural Only | NOT IMPLEMENTED | 0% | No projection engine. Queries go directly to SQLite. |
| Content-Addressed Store | Architectural Only | CONTRADICTS SPEC | 10% | FNV-1a 32-bit used instead of SHA-256. |
| Merkle Mountain Range | Architectural Only | CONTRADICTS SPEC | 10% | Binary Merkle tree rebuilt on insertion, not MMR. |
| ZK Proof System | Architectural Only | CONTRADICTS SPEC | 0% | Math.random() strings presented as proofs. |
| Ed25519 Signatures | Not Present | NOT IMPLEMENTED | 0% | No signature code exists anywhere in the repository. |
| RFC8785 Canonicalizer | Not Present | NOT IMPLEMENTED | 0% | No canonicalization logic. JSON.stringify used. |
| Schema Registry | Architectural Only | NOT IMPLEMENTED | 5% | No schema registry or versioning system. |
| Policy Engine (EPD) | Implemented | IMPLEMENTED | 80% | Tokenizer, parser, validator, evaluator, self-repair. |
| Policy Time-Travel | Architectural Only | PARTIALLY IMPLEMENTED | 15% | UI exists, no actual historical replay. |
| DAG Topology | Architectural Only | PARTIALLY IMPLEMENTED | 20% | Visualization exists, no real DAG structure. |
| Invariant Miner | Architectural Only | PARTIALLY IMPLEMENTED | 10% | UI component exists, no mining algorithm. |

| Spec Component | Architecture | Implementation | Ready | Notes |
|---|---|---|---|---|
| Shadow Bridge | Architectural Only | NOT IMPLEMENTED | 5% | Mock API only. No real shadow mode. |
| Federation Layer | Architectural Only | NOT IMPLEMENTED | 5% | UI exists, no federation protocol. |
| CLI Binary | Architectural Only | PARTIALLY IMPLEMENTED | 20% | EPD CLI exists for policy evaluation only. |
| Adapter System | Not Present | NOT IMPLEMENTED | 0% | No adapter abstraction layer. |
| Consensus Protocol | Not Present | NOT IMPLEMENTED | 0% | No consensus mechanism. |
| Verification Layer | Architectural Only | HARDCODED | 2% | verified: true with no actual verification. |

## Summary Statistics

The specification mapping reveals a stark distribution across the 20 components defined in the v0.8 specification. The overwhelming majority fall into the 'Architectural Only' category, meaning UI components or database schemas exist but no functional implementation backs them.

| Status Category | Count | Components |
|---|---|---|
| ARCHITECTURAL ONLY | 11 | Acceptance Pipeline, Fact Log, Sequencer, Projection, Content Store, MMR, ZK Proofs, Schema Registry, Policy Time-Travel, DAG Topology, Invariant Miner |
| CONTRADICTS SPEC | 3 | Content-Addressed Store (FNV-1a), MMR (binary Merkle), ZK Proofs (random strings) |
| PARTIALLY IMPLEMENTED | 3 | Shadow Bridge, Federation, CLI Binary |
| IMPLEMENTED | 1 | Policy Engine (EPD) |
| NOT PRESENT | 2 | Ed25519 Signatures, RFC8785 Canonicalizer |

**CONVERGENCE BLOCKER**

Three components do not merely fail to implement the specification; they actively contradict it. These contradictions must be resolved before any convergence trajectory can be established. Adding more dashboard sections on top of contradictory foundations will only increase the cost of correction.

# 4 Drift Detection Report

Drift detection identifies areas where the implementation has diverged from the specification. Unlike gaps (where something is missing), drifts represent active divergences where the implementation has taken a different path than the one specified. Each drift is assessed for its impact on system integrity, compliance, and the feasibility of reconciliation.

| # | Drift | Impact | Description | Reconciliation |
|---|---|---|---|---|
| 1 | No Acceptance Pipeline | CRITICAL | All writes bypass canonicalization, verification, and policy evaluation. No fact can be trusted. | Implement the full acceptance pipeline as the foundational layer. |
| 2 | Non-deterministic Hashing | CRITICAL | FNV-1a is non-cryptographic and produces collisions at scale. Content addressing is unreliable. | Replace FNV-1a with SHA-256 across all hash operations. |
| 3 | No Event Store | CRITICAL | No append-only log exists. State is mutable via Prisma CRUD. No audit trail or replay capability. | Implement Fact Log with append-only semantics. |
| 4 | Binary Merkle vs MMR | HIGH | Binary Merkle tree rebuilt on insertion loses historical proofs. Cannot verify past states. | Replace with proper Merkle Mountain Range implementation. |
| 5 | Fabricated ZK Proofs | CRITICAL | Math.random() strings presented as cryptographic proofs. Security vulnerability if deployed. | Remove fake proofs. Implement real ZK proof generation or stub honestly. |
| 6 | No Canonicalization | HIGH | JSON.stringify used for serialization. Key ordering is non-deterministic across engines. | Implement RFC8785 JSON Canonicalization Scheme. |
| 7 | Hardcoded Verification | HIGH | verified: true returned with no actual verification. False sense of security. | Implement real verification against signatures and proofs. |
| 8 | Date.now() in Kernel Code | HIGH | 5+ instances of Date.now() in code that should be deterministic. Timestamps vary on replay. | Replace with deterministic tick counter or logical clock. |
| 9 | No Schema Versioning | MEDIUM | No schema registry or version negotiation. Schema changes break existing policies silently. | Implement Schema Registry with version negotiation. |

## Drift Impact Distribution

Of the nine identified drifts, four are rated CRITICAL, meaning they represent fundamental architectural failures that prevent the system from fulfilling its specified purpose. Three are rated HIGH, meaning they undermine specific guarantees but do not entirely prevent operation. Two are rated MEDIUM, meaning they create operational risks but do not immediately break core functionality. No drifts are rated LOW; every identified drift requires remediation.

**HIGHEST-PRIORITY DRIFT**

The absence of an acceptance pipeline is the single most damaging drift. Every other drift either flows from this absence (no canonicalization, no verification, no signature checks) or is exacerbated by it (fabricated proofs go unchecked, non-deterministic hashes go unnoticed). Implementing the acceptance pipeline would create the structural backbone needed to address the remaining drifts.

# 5 Cryptographic Verification

The Epistemic Runtime v0.8 specification defines a comprehensive cryptographic foundation for ensuring data integrity, proof verification, and trust establishment. This chapter audits each cryptographic primitive against its specified requirement and the actual implementation.

## RFC8785: JSON Canonicalization

Status: NOT IMPLEMENTED. The specification requires that all facts undergo RFC8785 canonicalization before hashing to ensure deterministic serialization across different JSON libraries and runtime environments. The implementation uses JavaScript's built-in `JSON.stringify()`, which does not guarantee key ordering across different engines. This means the same logical fact can produce different hash values depending on the runtime, rendering content addressing unreliable and proof verification impossible across node boundaries.

## SHA-256: Content-Addressed Hashing

Status: CONTRADICTS SPEC. The specification mandates SHA-256 for all content-addressed hashing operations. The implementation uses FNV-1a 32-bit, a fast but non-cryptographic hash function designed for hash tables, not content addressing. FNV-1a produces only 32-bit hashes, making collisions statistically inevitable at volumes as low as tens of thousands of entries. SHA-256 produces 256-bit hashes with collision resistance sufficient for any foreseeable scale. This is not a performance optimization; it is a fundamental architectural error that compromises the integrity of the entire content-addressed storage layer.

## Ed25519: Digital Signatures

Status: NOT IMPLEMENTED. The specification requires Ed25519 signatures for fact authentication, ensuring that each fact can be attributed to a known identity and that tampering is detectable. No signature code exists anywhere in the repository. There is no key generation, no signing, no verification, and no key management infrastructure. Without signatures, there is no way to establish trust in the origin of any fact, and the trust runtime operates on hardcoded assumptions rather than cryptographic proof.

## MMR: Merkle Mountain Range

Status: CONTRADICTS SPEC. The specification requires a Merkle Mountain Range for efficient append-only proof generation and historical verification. The implementation uses a simple binary Merkle tree that is fully rebuilt on every insertion. This approach has three critical failures: it is computationally expensive (O(n) per insertion instead of O(log n)), it destroys historical proof capability (previous root hashes become invalid), and it cannot produce membership proofs for arbitrary historical states. The MMR is fundamental to the specification's guarantee of incremental verifiability.

## ZK Proofs: Zero-Knowledge Verification

Status: FABRICATED. The specification describes zero-knowledge proof generation for policy compliance verification, allowing parties to prove compliance without revealing underlying data. The implementation generates random hexadecimal strings using `Math.random().toString(16)` and presents them as proof values. These are not proofs; they are random strings with zero cryptographic validity. If this system were deployed, an attacker could generate arbitrary 'proofs' using the same Math.random() approach and the system would accept them as valid.

## Verification Status

Status: HARDCODED. The verification layer returns `verified: true` for all entities without performing any actual verification. There is no signature checking, no proof validation, no hash comparison, and no policy compliance check. The verification field in the database and API responses is decorative, not functional.

**CRYPTOGRAPHIC SUMMARY**

Of six cryptographic requirements, zero are correctly implemented. Three are contradicted (SHA-256, MMR, ZK Proofs), two are not implemented at all (RFC8785, Ed25519), and one is hardcoded (Verification). The cryptographic foundation of the specification is entirely absent.

# 6 Acceptance Pipeline Verification

The acceptance pipeline is the architectural backbone of the Epistemic Runtime. According to the v0.8 specification, every write operation must pass through the acceptance pipeline before being committed to the fact log. The pipeline performs five critical functions: canonicalization, schema verification, signature validation, policy evaluation, and deterministic sequencing. This chapter verifies whether the acceptance pipeline exists and operates as specified.

## Pipeline Existence

The acceptance pipeline does not exist. There is no module, class, or function in the codebase that implements any part of the acceptance pipeline. The API route at `/api/acceptance-engine` is a placeholder that returns an empty response. The component at `src/components/epistemic/acceptance-engine.tsx` is a dashboard visualization that displays a mock pipeline UI with no connection to any backend pipeline logic.

## Data Flow Analysis

Every write operation in the system follows the same path: the browser sends a request to an API route, the API route calls Prisma, Prisma writes directly to SQLite. There is no intermediate processing layer. The following table shows the analysis of each write-capable API route:

| Route | Write Method | Pipeline Stage | Verification |
|---|---|---|---|
| POST /api/policies | prisma.policy.create() | NONE | No canonicalization, no signature |
| PUT /api/policies/[id] | prisma.policy.update() | NONE | No schema check, no policy eval |
| DELETE /api/policies/[id] | prisma.policy.delete() | NONE | No tombstone, no audit trail |
| POST /api/merges/simulate | Hardcoded response | NONE | Not a real operation |
| POST /api/acceptance-engine | Empty handler | NONE | Route exists but does nothing |

## Missing Pipeline Stages

The specification defines five stages that must execute in order before any fact is committed. None of these stages exist in the current implementation:

- Canonicalization: Facts must be serialized using RFC8785 to produce a deterministic byte representation. Currently, facts are stored as-is with no serialization guarantee.

- Schema Verification: Facts must be validated against a registered schema before acceptance. There is no schema registry and no validation against any schema definition.
- Signature Validation: Facts must carry an Ed25519 signature from an authorized identity. No signature mechanism exists in the codebase.
- Policy Evaluation: Facts must pass through the EPD policy engine for compliance checking. While the EPD engine exists, it is never invoked during the write path.
- Deterministic Sequencing: Accepted facts must be assigned a deterministic sequence number. Facts currently use database auto-increment IDs, which are non-deterministic across instances.

**PIPELINE VERDICT**

The acceptance pipeline is entirely absent. Every write operation in the system bypasses all specified safety mechanisms. Three API routes return hardcoded mock data and perform no real operations at all. The system provides no guarantees about data integrity, provenance, or policy compliance.

# 7 Determinism Audit

Determinism is a non-negotiable property of the Epistemic Runtime. The specification requires that any fact, given the same inputs, must produce the same output at any point in time and on any node. This audit identifies every instance of non-determinism in the codebase, classified by severity.

## CRITICAL Non-Determinism Instances

The following instances represent fundamental violations of the determinism requirement. Each one can cause the same logical operation to produce different results on different executions, nodes, or replay attempts.

| # | Instance | Location | Description |
|---|---|---|---|
| 1 | Date.now() | src/lib/epd/index.ts | The now() function in the EPD evaluator returns Date.now(), making evaluation results time-dependent. Two evaluations of the same policy at different times produce different timestamps. |
| 2 | Date.now() | src/lib/seed.ts | Seed data uses Date.now() for timestamp fields, making the seed non-reproducible across runs. |
| 3 | Date.now() | src/app/api/policies/route.ts | Policy creation timestamps use Date.now(), ensuring the same policy created twice gets different timestamps. |
| 4 | Date.now() | src/app/api/merges/route.ts | Merge operations use Date.now() for merge timestamps, making merge results non-reproducible. |
| 5 | Date.now() | src/app/api/audit/route.ts | Audit log entries use Date.now(), making audit trails non-deterministic. |
| 6 | Math.random() | src/app/api/proofs/route.ts | ZK 'proofs' are generated using Math.random(), producing different values on every request. |
| 7 | Math.random() | src/components/epistemic/zk-circuit.tsx | Frontend ZK circuit visualization uses Math.random() for proof display values. |
| 8 | JSON.stringify() | src/lib/epd/index.ts | Hash computation uses JSON.stringify() for serialization, which does not guarantee key ordering across JavaScript engines or even across V8 versions. |

## HIGH Non-Determinism Instances

| # | Instance | Location | Description |
|---|---|---|---|
| 1 | Math.random() | src/components/epistemic/gossip-sim.tsx | Gossip simulation uses Math.random() for network delay and peer selection simulation. |
| 2 | Math.random() | src/components/epistemic/shard-rebalance.tsx | Shard rebalance simulation uses Math.random() for shard assignment. |

| # | Instance | Location | Description |
|---|---|---|---|
| 3 | Math.random() | src/components/epistemic/invariant-miner.tsx | Invariant mining visualization uses Math.random() for candidate selection display. |

## Impact Summary

The determinism audit reveals 8 CRITICAL and 3 HIGH instances of non-determinism. The most damaging pattern is the use of `Date.now()` in what should be kernel code. The EPD evaluator's `now()` function is particularly concerning because it means policy evaluation results depend on wall-clock time, making replay impossible. The use of `Math.random()` in the proofs API route means that the same query returns different 'proofs' on every request, which would be immediately detectable in any real deployment.

The `JSON.stringify()` usage for hashing is a subtle but critical issue. JavaScript does not guarantee key ordering in object serialization. Two logically identical facts with keys in different insertion order will produce different string representations and therefore different hashes. This breaks content addressing at a fundamental level.

**DETERMINISM VERDICT**

The system is fundamentally non-deterministic. No replay, no verification, and no consensus is possible when the same inputs produce different outputs. The 8 CRITICAL instances must all be resolved before the system can be considered even minimally compliant with the specification.

# 8 Technical Debt Register

Technical debt is categorized by domain to provide a structured view of what must be addressed to achieve convergence with the specification. Each debt item represents work that must be completed before the system can operate as specified. The debt is substantial and spans every architectural layer of the system.

## Architecture Debt

- No Fact Model: The specification defines facts as the fundamental unit of knowledge, with strict typing, canonicalization, and content addressing. The implementation has no fact model. Data is stored as generic Prisma models with no structural guarantees.
- No Event Sourcing: The specification mandates an append-only fact log as the source of truth. The implementation uses mutable CRUD operations on SQLite. Records can be updated and deleted, destroying the audit trail.
- No Acceptance Pipeline: As detailed in Chapter 6, the acceptance pipeline is entirely absent. This is the single largest architectural debt item, as the pipeline is the structural backbone of the specification.
- No Projection Engine: The specification requires projections to derive current state from the fact log. Without a fact log, projections are impossible. Queries go directly to SQLite tables.

## Implementation Debt

- FNV-1a Hash: A 32-bit non-cryptographic hash function is used where SHA-256 is specified. This produces collisions at scale and provides no integrity guarantees.
- Binary Merkle Tree: A simple binary Merkle tree that rebuilds on every insertion is used where a Merkle Mountain Range is specified. This destroys historical proof capability.
- Fake ZK Proofs: Random strings generated by Math.random() are presented as zero-knowledge proofs. This is not a shortcut; it is a fabrication that must be removed.
- Hardcoded Verification: The verification status of all entities is hardcoded to true. No actual verification is performed.

## Infrastructure Debt

- No Adapters: The specification defines an adapter abstraction for connecting external data sources. No adapter system exists.
- No Consensus: The specification describes a consensus protocol for multi-node agreement. No consensus mechanism exists.

- No CLI Binary: The EPD CLI tool exists but only handles policy evaluation. No kernel CLI binary exists for administrative operations.

## Testing Debt

- No Test Suite: The repository has zero test files. There are no unit tests, no integration tests, no property-based tests, and no end-to-end tests. Every aspect of the system is untested.
- No CI Pipeline: There is no continuous integration pipeline. No automated checks run on commits or pull requests.

## Security Debt

- No Cryptographic Verification: As detailed in Chapter 5, no cryptographic primitives are correctly implemented. The system has no integrity verification mechanism.
- Fake Proofs: The fabricated ZK proofs represent a security vulnerability. If deployed, they would provide a false sense of security while offering no actual protection.
- No Signature Validation: Without Ed25519 signatures, there is no way to verify the origin of any fact. Any client can inject arbitrary data.

## Operational Debt

- No Deployment Automation: There is no infrastructure-as-code, no containerization, and no deployment pipeline.
- No Monitoring: There are no health checks, no metrics collection, and no alerting. The system has no observability.
- No Runbook: There are no operational procedures documented for incident response, backup, or recovery.

# 9 Convergence Assessment

The central question of this audit is whether the implementation is converging toward the Epistemic Runtime v0.8 specification. Convergence means that with each iteration, the implementation moves closer to the specified architecture, replacing mock data with real logic, adding missing components, and resolving contradictions. Divergence means the implementation is moving away from the specification, adding features that are not in the spec while neglecting those that are.

## Current Trajectory: Diverging

The evidence from this audit is clear: the implementation is not converging toward the specification. The current development trajectory adds more dashboard sections and mock API routes, not kernel implementation. Each new visualization component makes the dashboard more impressive while leaving the kernel layer unchanged. The gap between specification and implementation is growing, not shrinking.

This is not a criticism of the dashboard work -- the dashboard is excellent at approximately 90% completeness. The problem is that the dashboard is a visualization layer for a kernel that does not exist. A perfect dashboard on top of an empty kernel is still an empty kernel. The dashboard shows what the system would look like if it worked; it does not make the system work.

## The Dashboard-Kernel Imbalance

| Layer | Completeness | Assessment |
|---|---|---|
| Dashboard (UI Components) | ~90% | Well-structured, visually complete, good UX |
| API Layer (Routes) | ~60% | Most routes functional, some mock, some empty |
| Kernel (Core Logic) | ~5% | Only EPD engine qualifies as kernel code |
| Infrastructure | ~2% | No adapters, no consensus, no CLI |

## What Convergence Would Look Like

True convergence would require the development trajectory to shift from dashboard-first to kernel-first. The single highest-impact action would be implementing the acceptance pipeline. This would establish the structural backbone that all other kernel components depend on. Without the acceptance pipeline, implementing the fact log, the deterministic sequencer, and the projection engine would have no integration point.

Convergence would be measurable by tracking the following indicators over time: the percentage of API routes that route through the acceptance pipeline (currently 0%), the number of mock API routes replaced with real

implementations (currently 0 of 3), the resolution of spec contradictions (currently 0 of 3), and the reduction in non-determinism instances (currently 0 of 11). All four indicators are at zero, confirming that convergence has not begun.

**CONVERGENCE VERDICT**

The implementation is NOT converging toward the specification. The development trajectory is adding visualization layers, not kernel logic. Without implementing the acceptance pipeline as the foundational layer, the project will continue to diverge. The dashboard is excellent; the kernel is nearly empty. Success requires pivoting from dashboard-first to kernel-first development immediately.

# 10 Execution Roadmap

This roadmap defines ten implementation steps that would move the project from its current state toward specification compliance. Steps are ordered by dependency: foundational steps must be completed before dependent steps can begin. Each step includes its purpose, dependencies, affected modules, complexity assessment, and acceptance criteria.

## Step 1: Complete Acceptance Pipeline

Purpose: Establish the structural backbone of the kernel. The acceptance pipeline mediates all write operations, ensuring that every fact passes through canonicalization, schema verification, signature validation, and policy evaluation before being committed to the fact log.

Dependencies: None. This is the foundational step.

Affected Modules: All API routes, Prisma write operations, EPD evaluator integration

Complexity: HIGH. Requires implementing the full pipeline with five stages, error handling, rejection logging, and integration with the existing EPD evaluator.

Acceptance Criteria: All write operations route through the pipeline. Rejected facts are logged with reasons. Accepted facts carry canonical form, schema hash, and evaluation result.

## Step 2: Replace FNV-1a with SHA-256

Purpose: Ensure content-addressed hashing meets the specification's cryptographic requirements. SHA-256 provides 256-bit hashes with sufficient collision resistance for any foreseeable scale, replacing the 32-bit FNV-1a hashes that produce collisions at volume.

Dependencies: None. Can be implemented independently.

Affected Modules: All hash computation code, Merkle tree construction, content-addressed storage, API routes that compute or return hashes

Complexity: MEDIUM. Straightforward replacement but requires database migration for existing hash values.

Acceptance Criteria: All hash operations produce SHA-256 digests. Existing data is rehashed during migration. All tests pass with new hash values.

## Step 3: Implement RFC8785 Canonicalizer

Purpose: Ensure deterministic JSON serialization across all runtime environments. RFC8785 defines a canonical JSON serialization that produces identical byte representations regardless of key insertion order,

whitespace preferences, or number formatting differences.

Dependencies: None. Can be implemented independently.

Affected Modules: Acceptance pipeline (canonicalization stage), hash computation, fact serialization, signature payload construction

Complexity: MEDIUM. Requires implementing the JCS algorithm or integrating a proven library, plus replacing all JSON.stringify() calls in hash computation paths.

Acceptance Criteria: The same logical object always produces the same canonical byte representation, regardless of key ordering in the source object.

## Step 4: Implement Fact Log

Purpose: Replace mutable CRUD with an append-only event store. The fact log is the source of truth, recording every accepted fact in immutable, sequenced order. Projections derive current state by replaying the fact log.

Dependencies: Step 1 (Acceptance Pipeline). Facts must be accepted before they are logged.

Affected Modules: All write operations, Prisma schema (new Fact table), API routes, projection queries

Complexity: HIGH. Requires a new database schema, migration of existing data to fact format, and restructuring all read paths to use projections instead of direct queries.

Acceptance Criteria: All accepted facts are recorded in the fact log. No update or delete operations exist on the fact table. Current state is derived from projections.

## Step 5: Implement Deterministic Sequencer

Purpose: Assign deterministic sequence numbers to accepted facts. The sequencer replaces database auto-increment IDs with a deterministic ordering scheme that produces identical sequences across nodes given the same input order.

Dependencies: Step 4 (Fact Log). The sequencer assigns positions within the fact log.

Affected Modules: Fact log writes, all references to record IDs, consensus protocol (future), replication (future)

Complexity: MEDIUM. Requires a deterministic ordering algorithm (e.g., hybrid logical clocks or Lamport timestamps) and migration of existing ID references.

Acceptance Criteria: The same set of facts, received in the same order, always produces the same sequence numbers on any node.

## Step 6: Implement Projection Engine

Purpose: Derive current state from the fact log by replaying events through projection functions. Projections replace direct database queries with computed views that are always consistent with the fact log.

Dependencies: Step 4 (Fact Log). Projections read from the fact log.

Affected Modules: All read operations, dashboard data sources, API read routes, search and filter logic

Complexity: HIGH. Requires implementing projection functions for every current query, plus a projection registry and incremental update mechanism.

Acceptance Criteria: All read operations use projections. Projections are automatically updated when new facts are appended. Point-in-time queries are supported.

## Step 7: Implement Ed25519 Signatures

Purpose: Enable cryptographic authentication of facts. Each fact carries an Ed25519 signature from the identity that authored it, allowing verification of fact provenance and detection of tampering.

Dependencies: Step 3 (RFC8785 Canonicalizer). Signatures are computed over canonical fact representations.

Affected Modules: Acceptance pipeline (signature validation stage), fact creation, trust runtime, verification layer

Complexity: MEDIUM. Requires Ed25519 library integration, key management, and signature verification in the acceptance pipeline.

Acceptance Criteria: Every accepted fact carries a valid Ed25519 signature. The acceptance pipeline rejects unsigned or invalidly signed facts.

## Step 8: Fix MMR Implementation

Purpose: Replace the simple binary Merkle tree with a proper Merkle Mountain Range. The MMR supports efficient append-only proof generation, historical verification, and incremental root updates without rebuilding the entire tree.

Dependencies: Step 2 (SHA-256). MMR nodes must use SHA-256 hashes.

Affected Modules: Merkle tree construction, proof generation, verification layer, dashboard MMR visualization

Complexity: HIGH. Requires implementing the full MMR data structure with bagging peaks, proof generation, and verification. This is algorithmically non-trivial.

Acceptance Criteria: The MMR supports O(log n) append operations, membership proofs for any historical fact, and consistent root hashes across nodes.

## Step 9: Implement Schema Registry

Purpose: Enable schema versioning and validation. The schema registry stores versioned schema definitions and validates facts against their declared schema before acceptance.

Dependencies: Step 4 (Fact Log). Schemas are associated with fact types in the fact log.

Affected Modules: Acceptance pipeline (schema verification stage), fact creation, policy evaluation, EPD engine

Complexity: MEDIUM. Requires schema definition format, version negotiation protocol, and validation engine.

Acceptance Criteria: Facts are validated against registered schemas. Schema changes are versioned. Backward compatibility is maintained.

## Step 10: Implement Policy Time-Travel

Purpose: Enable evaluation of policies against historical states. Policy time-travel allows querying what a policy would have evaluated to at any point in the fact log's history, supporting audit, compliance, and debugging use cases.

Dependencies: Step 9 (Schema Registry). Historical schemas must be available for correct historical evaluation.

Affected Modules: EPD evaluator, projection engine, timeline API, dashboard timeline visualization

Complexity: MEDIUM. Requires point-in-time projection queries and EPD evaluator integration with historical fact contexts.

Acceptance Criteria: Any policy can be evaluated against any historical point in the fact log. Results are deterministic and reproducible.

## Roadmap Dependencies

```
Step 1: Acceptance Pipeline [no deps] --> foundational Step 2: SHA-256 [no deps] -->
independent Step 3: RFC8785 Canonicalizer [no deps] --> independent Step 4: Fact Log
[Step 1] --> depends on pipeline Step 5: Deterministic Sequencer [Step 4] -->
depends on fact log Step 6: Projection Engine [Step 4] --> depends on fact log Step
7: Ed25519 Signatures [Step 3] --> depends on canonicalizer Step 8: Fix MMR [Step 2]
--> depends on SHA-256 Step 9: Schema Registry [Step 4] --> depends on fact log Step
10: Policy Time-Travel [Step 9] --> depends on schema registry
```

Steps 1, 2, and 3 have no dependencies and can begin immediately in parallel. Steps 4-10 form a dependency chain that must be executed in sequence. The estimated timeline for full completion is 8-12 months of focused

kernel development with a team of at least two engineers dedicated to kernel work (not dashboard work).

# 11 Risk Register

The following risks are identified based on the current state of the implementation. Each risk is assessed for likelihood, impact, and the urgency of mitigation. Risks are ordered by severity.

## Risk 1: Dashboard-First Development Continues

Likelihood: HIGH. Impact: CRITICAL. Urgency: IMMEDIATE.

The current development pattern adds dashboard sections and mock APIs rather than kernel implementation. If this continues, the gap between specification and implementation will widen indefinitely. Each new mock section increases the cost of replacing it with real logic, because the dashboard integration points multiply. Mitigation: mandate kernel-first development with dashboard updates only when kernel functionality exists to support them.

## Risk 2: FNV-1a Hash Collisions at Scale

Likelihood: HIGH. Impact: CRITICAL. Urgency: HIGH.

FNV-1a 32-bit produces only 4,294,967,296 possible hash values. By the birthday paradox, collisions become likely at approximately 65,536 entries. In a production system with continuous writes, this threshold could be reached in days or hours. A collision would cause two different facts to be treated as identical, corrupting the content-addressed store. Mitigation: replace FNV-1a with SHA-256 as specified.

## Risk 3: Non-deterministic Replay

Likelihood: CERTAIN. Impact: CRITICAL. Urgency: HIGH.

The 8 CRITICAL non-determinism instances guarantee that replay will produce different results. This makes audit and compliance impossible. In regulated environments (which the specification explicitly targets), the inability to demonstrate deterministic replay is a compliance failure. Mitigation: eliminate all Date.now(), Math.random(), and JSON.stringify() usage in kernel code.

## Risk 4: Fabricated ZK Proofs as Security Vulnerability

Likelihood: HIGH (if deployed). Impact: CRITICAL. Urgency: HIGH.

If the system is deployed with fabricated ZK proofs, it will present a security interface that provides no actual security. An attacker can generate arbitrary 'proofs' using the same Math.random() approach and the system will accept them. This is not a theoretical risk; it is an active vulnerability that would be exploitable from day one. Mitigation: remove all fabricated proofs immediately and replace with honest stubs or real

implementation.

## Risk 5: No Test Coverage

Likelihood: CERTAIN. Impact: HIGH. Urgency: MEDIUM.

With zero test coverage, any change to the codebase risks introducing regressions that go undetected. As the kernel is implemented, the lack of tests will make it impossible to verify that new code does not break existing functionality. Mitigation: establish a test suite with at minimum unit tests for all kernel modules before beginning roadmap implementation.

# 12 Final Verdict

This convergence audit has examined the Epistemic Runtime repository against the v0.8 specification across twelve dimensions: specification mapping, drift detection, cryptographic verification, acceptance pipeline, determinism, technical debt, convergence trajectory, execution roadmap, and risk assessment. The verdict is based on four questions that determine whether the project is on track to deliver the specified system.

## Question 1: Are We Building the Same System?

**ANSWER: PARTIALLY**

The specification describes an Epistemic Runtime kernel: an acceptance pipeline, fact log, deterministic sequencer, projection engine, and cryptographic verification layer. The implementation is a dashboard that visualizes what such a kernel would look like. These are fundamentally different things. The dashboard is a presentation layer; the kernel is an execution engine. The only shared component is the EPD policy engine, which is real and well-implemented but operates in isolation from the rest of the system.

## Question 2: Is Implementation Converging?

**ANSWER: NO**

The development trajectory adds visualization components and mock API routes, not kernel logic. Each iteration makes the dashboard more impressive while leaving the kernel layer unchanged. The specification implementation percentage has not increased meaningfully. The gap between specification and implementation is growing, not shrinking. Convergence has not begun.

## Question 3: Are There Contradictions?

**ANSWER: YES — THREE CRITICAL CONTRADICTIONS**

1. Hash Algorithm: FNV-1a 32-bit instead of SHA-256. This is not a preference; it is a fundamental architectural divergence affecting data integrity. 2. Merkle Structure: Binary Merkle tree rebuilt on insertion instead of Merkle Mountain Range. This destroys historical proof capability. 3. ZK Proofs: Math.random() strings instead of cryptographic proofs. This is fabrication, not implementation.

## Question 4: What Is the Minimum Remaining Work?

The minimum remaining work to achieve specification compliance is defined by the 10-step execution roadmap in Chapter 10. This represents approximately 8-12 months of focused kernel development with a dedicated team. The three independent steps (Acceptance Pipeline, SHA-256, RFC8785 Canonicalizer) can begin immediately. The remaining seven steps form a dependency chain that must be executed in sequence.

| Step | Component | Dependencies | Complexity | Estimated Duration |
|---|---|---|---|---|
| 1 | Acceptance Pipeline | None | HIGH | 6-8 weeks |
| 2 | SHA-256 Hash | None | MEDIUM | 2-3 weeks |
| 3 | RFC8785 Canonicalizer | None | MEDIUM | 3-4 weeks |
| 4 | Fact Log | Step 1 | HIGH | 6-8 weeks |
| 5 | Deterministic Sequencer | Step 4 | MEDIUM | 3-4 weeks |
| 6 | Projection Engine | Step 4 | HIGH | 6-8 weeks |
| 7 | Ed25519 Signatures | Step 3 | MEDIUM | 3-4 weeks |
| 8 | MMR Implementation | Step 2 | HIGH | 6-8 weeks |
| 9 | Schema Registry | Step 4 | MEDIUM | 3-4 weeks |
| 10 | Policy Time-Travel | Step 9 | MEDIUM | 3-4 weeks |

## Closing Statement

**FROM HOPE TO PROOF**

The Epistemic Runtime v0.8 specification describes a system that provides cryptographic proof of knowledge integrity. The current implementation provides visualizations of what such proofs might look like. The distance between these two things is the distance between hope and proof. Closing this gap requires a fundamental shift in development priorities: from dashboard-first to kernel-first. The dashboard is excellent. The kernel is nearly empty. Success requires building the kernel that the dashboard promises. The roadmap exists. The specification is clear. What remains is the discipline to execute.